Dependency Injection

```
Public class User {
    Order order = new Order();
    Public User() {
        S.o.p("initializing user");
    }
}
```

```
Public class Order {
    Public Order() {
        S.o.p("initializing Order");
    }
}
```

Issues with above class structure;

1:- Both User and Order class are Tightly coupled.

* Suppose, Order object ~~class~~ creation logic get changed (let say in future object becomes an interface and it has many concrete class), then User class has to be changed too.

```
Public class User() {
    Order order = new Order();
                  new OnlineOrder();
    Public User() {
        S.o.p("in");
    }
}
```

```
Public interface Order {
}
```

```
Public class OnlineOrder implement
                Order {
}
```

```
Public class OfflineOrder implement
                Order {
}
```

Concept && Coding

2:- It breaks Dependency Inversion rule of SOLID principle. (2)

(i) This principle say that DO NOT depend on concrete implementation, rather depends on abstraction.

```
Public class User {
    Order order = new OnlineOrder();
    Public User() {
        S.o.p ("initializing user");
    }
}
```

Breaks Dependency Inver-sion on Princip

```
Public class User {
    Order order;
    Public User (Order orderobj)
        this.order = orderobj;
    }
}
```

Follows Dependency Inversio

Concept && Coding